

Part 1: Key Identification Exercises

Task 1.1: Superkey and Candidate Key Analysis

Given the following relations, identify all superkeys and candidate keys:

Relation A: Employee

Employee(EmpID, SSN, Email, Phone, Name, Department, Salary)

Sample Data:

EmpID	SSN	Email	Phone	Name	Department	Salary
101	123-45-6789	john@company.com	555-0101	John	IT	75000
102	987-65-4321	mary@company.com	555-0102	Mary	HR	68000
103	456-78-9123	bob@company.com	555-0103	Bob	IT	72000

Your Tasks: 1. List at least 6 different superkeys 2. Identify all candidate keys 3. Which candidate key would you choose as primary key and why? 4. Can two employees have the same phone number? Justify your answer based on the data shown.

- 1) {EmpID}, {SSN}, {email}, {Phone}, {EmpID, Name}, {SSN, Department}
- 2) {EmpID}, {SSN}, {Email}, {Phone}
- 3) {EmpID}. Because it is surrogate key, and the other ones either can be changed (SSN, Email, Phone) or are not unique
- 4) No. As seen in the example, all phone numbers are identified uniquely: the last 4 digits are an increment, so the phone number is unique for each employee.

Relation B: Course Registration

Registration(StudentID, CourseCode, Section, Semester, Year, Grade, Credits)

Business Rules:

- A student can take the same course in different semesters
- A student cannot register for the same course section in the same semester
- Each course section in a semester has a fixed credit value

Your Tasks: 1. Determine the minimum attributes needed for the primary key 2. Explain why each attribute in your primary key is necessary 3. Identify any additional candidate keys (if they exist)

1) {StudentID}

2) StudentID must be surrogate key that is an increment, that is enough for a primary key

3) {Section, Semester}, {Section, Credits}

Task 1.2: Foreign Key Design

Design the foreign key relationships for this university system:

Given Tables:

Student(StudentID, Name, Email, Major, AdvisorID)

Professor(ProfID, Name, Department, Salary)

Course(CourseID, Title, Credits, DepartmentCode)

Department(DeptCode, DeptName, Budget, ChairID)

Enrollment(StudentID, CourseID, Semester, Grade)

Your Tasks: 1. Identify all foreign key relationships

- 1) Student --- {StudentID} - {StudentID} --- Enrollment
- 2) Course --- {DepartmentCode} - {DeptCode} --- Department
- 3) Course --- {CourseID} - {CourseID} --- Enrollment
- 4) Professor --- {Department} - {DeptName} --- Department

Part 2: ER Diagram Construction

Task 2.1: Hospital Management System

Scenario: Design a database for a hospital management system.

Requirements:

- **Patients** have unique patient IDs, names, birthdates, addresses (street, city, state, zip), phone numbers (multiple allowed), and insurance information
- **Doctors** have unique doctor IDs, names, specializations (can have multiple), phone numbers, and office locations
- **Departments** have department codes, names, and locations
- **Appointments** track which patient sees which doctor at what date/time, the purpose of visit, and any notes
- **Prescriptions** track medications prescribed by doctors to patients, including dosage and instructions
- **Hospital Rooms** are numbered within departments (room 101 in Cardiology is different from room 101 in Neurology)

Your Tasks: 1. Identify all entities (specify which are strong and which are weak) 2. Identify all attributes for each entity (classify as simple, composite, multi-valued, or derived) 3. Identify all relationships with their cardinalities (1:1, 1:N, M:N) 4. Draw the complete ER diagram using proper notation 5. Mark primary keys

1) Patients (strong), Doctors (strong), Departments (strong), Appointments (weak), Prescriptions (weak), HospitalRooms (strong)

2) Patients {PatientID (simple), Name (simple), Birthdate (simple), Address (composite), Phones (multivalued), Insurance (simple)}

Doctors {DoctorID (simple), Name (simple), Specialization (multivalued), Phone (simple), Office (composite)}

Departments {DeptCode (simple), Name (simple), Location (simple)}

Appointments {PatientID (simple), DoctorID (simple), DateTime (composite), VisitInfo (simple), Notes (simple)}

Prescriptions {PatientID (simple), DoctorID (simple), Medications (composite)}

HospitalRooms {RoomID (simple), DeptCode (simple), DoctorID (simple), Room (simple)}

Part 2: ER Diagram Construction

Task 2.1: Hospital Management System

Scenario: Design a database for a hospital management system.

Requirements:

- **Patients** have unique patient IDs, names, birthdates, addresses (street, city, state, zip), phone numbers (multiple allowed), and insurance information
- **Doctors** have unique doctor IDs, names, specializations (can have multiple), phone numbers, and office locations
- **Departments** have department codes, names, and locations
- **Appointments** track which patient sees which doctor at what date/time, the purpose of visit, and any notes
- **Prescriptions** track medications prescribed by doctors to patients, including dosage and instructions
- **Hospital Rooms** are numbered within departments (room 101 in Cardiology is different from room 101 in Neurology)

Your Tasks: 1. Identify all entities (specify which are strong and which are weak) 2. Identify all attributes for each entity (classify as simple, composite, multi-valued, or derived) 3. Identify all relationships with their cardinalities (1:1, 1:N, M:N) 4. Draw the complete ER diagram using proper notation 5. Mark primary keys

3) Patients --- {PatientID} - Appointments - {DoctorID} --- Doctors (M:N)
Patients --- {PatientID} - Prescriptions - {DoctorID} --- Doctors (M:N)
Doctors --- {Room} - {Room} --- HospitalRooms --- {DeptCode} - {DeptCode} --- Departments (M:N)

4) As seen in the corresponded file alongside the pdf file

Task 2.2: E-commerce Platform

Scenario: Design a simplified e-commerce database.

Requirements:

- **Customers** place **Orders** for **Products**
- **Products** belong to **Categories** and are supplied by **Vendors**
- **Orders** contain multiple **Order Items** (quantity and price at time of order)
- **Products** have reviews and ratings from customers
- Track **Inventory** levels for each product
- **Shipping addresses** can be different from customer billing addresses

Your Tasks: 1. Create a complete ER diagram 2. Identify at least one weak entity and justify why it's weak 3. Identify at least one many-to-many relationship that needs attributes

1) As seen in the corresponded file

2) Orders. It does not need a primary key because it is a table made via M:N connection, so identification goes by Customers and Products primary keys, which means those tables are the owners of the Orders table. Therefore, it is a weak entity.

3) Customers --- {CustomerID} - {CustomerID} --- Orders --- {ProductID} - {ProductID} --- Products

Part 4: Normalization Workshop

Task 4.1: Denormalized Table Analysis

Given Table:

StudentProject(StudentID, StudentName, StudentMajor, ProjectID, ProjectTitle,
ProjectType, SupervisorID, SupervisorName, SupervisorDept,
Role, HoursWorked, StartDate, EndDate)

Your Tasks: 1. **Identify functional dependencies:** List all FDs in the format $A \rightarrow B$ 2. **Identify problems:** - What redundancy exists in this table? - Give specific examples of update, insert, and delete anomalies 3. **Apply 1NF:** Are there any 1NF violations? How would you fix them? 4. **Apply 2NF:** - What is the primary key of this table? - Identify any partial dependencies - Show the 2NF decomposition 5. **Apply 3NF:** - Identify any transitive dependencies - Show the final 3NF decomposition with all table schemas

1) StudentID $\rightarrow$ StudentName, StudentMajor
ProjectID $\rightarrow$ ProjectTitle, ProjectType
SupervisorID $\rightarrow$ SupervisorName, SupervisorDept

2) To insert an information about a supervisor, it is required to tie them with a student and a project, so it is needed to also insert info about the student and the project. That is an insertion anomaly.

To update an information about a project, it is required to update several rows for the students whose project it that one, which creates a redundancy. That is an update anomaly.

To delete a student from the table, it is needed to delete an entire row, which may contain an information about the project and the supervisor that should not be deleted. That is a delete anomaly.

3) No

Part 4: Normalization Workshop

Task 4.1: Denormalized Table Analysis

Given Table:

StudentProject(StudentID, StudentName, StudentMajor, ProjectID, ProjectTitle,
ProjectType, SupervisorID, SupervisorName, SupervisorDept,
Role, HoursWorked, StartDate, EndDate)

Your Tasks: 1. **Identify functional dependencies:** List all FDs in the format $A \rightarrow B$ 2. **Identify problems:** - What redundancy exists in this table? - Give specific examples of update, insert, and delete anomalies 3. **Apply 1NF:** Are there any 1NF violations? How would you fix them? 4. **Apply 2NF:** - What is the primary key of this table? - Identify any partial dependencies - Show the 2NF decomposition 5. **Apply 3NF:** - Identify any transitive dependencies - Show the final 3NF decomposition with all table schemas

4) {StudentID, ProjectID, SupervisorID}
StudentID $\rightarrow$ StudentName, StudentMajor
ProjectID $\rightarrow$ ProjectTitle, ProjectType
SupervisorID $\rightarrow$ SupervisorName, SupervisorDept

Student {StudentID, StudentName, StudentMajor}
Project {ProjectID, ProjectTitle, ProjectType}
Supervisor {SupervisorID, SupervisorName, SupervisorDept}

StudentProject {StudentID, ProjectID, SupervisorID, Role, HoursWorked, StartDate, EndDate}

5) There is none transitive dependencies, so the 2NF is already a 3NF.

Task 4.2: Advanced Normalization

Given Table:

CourseSchedule(StudentID, StudentMajor, CourseID, CourseName,
InstructorID, InstructorName, TimeSlot, Room, Building)

Business Rules:

- Each student has exactly one major
- Each course has a fixed name
- Each instructor has exactly one name
- Each time slot in a room determines the building (rooms are unique across campus)
- Each course section is taught by one instructor at one time in one room
- A student can be enrolled in multiple course sections

Your Tasks: 1. Determine the primary key of this table (hint: this is tricky!) 2. List all functional dependencies 3. Check if the table is in BCNF 4. If not in BCNF, decompose it to BCNF showing your work 5. Explain any potential loss of information in your decomposition

1) {StudentID, CourseID, TimeSlot}

2) StudentID → StudentMajor

CourseID → CourseName

InstructorID → InstructorName

Room → Building

{CourseID, TimeSlot} → InstructorID, Room

3) No

4) Student {StudentID PK, StudentMajor}

Course {CourseID PK, CourseName}

Instructor {InstructorID PK, InstructorName}

Room {Room PK, Building}

CourseSection {Course ID FK, TimeSlot PK,
InstructorID FK, Room FK}

Enrollment {StudentID FK, CourseID FK, TimeSlot FK}

Student --- Enrollment --- CourseSection

Course --- CourseSection

Instructor ---- CourseSection

Room --- CourseSection

5) The idea was to connect Course, Instructor and Room Tables to the CourseSection Table as 1:N, so the student table was separated from the others. Then, we can connect the Student table to CourseSection as M:N by adding a third table.

Thus, there cannot be a potential loss of information, and the original forms can be returned by joining either on CourseSection or Enrollment keys

Part 5: Design Challenge

Task 5.1: Real-World Application

Scenario: Your university wants to track student clubs and organizations with the following requirements:

System Requirements:

- Student clubs and organizations information
- Club membership (students can join multiple clubs, clubs have multiple members)
- Club events and student attendance tracking
- Club officer positions (president, treasurer, secretary, etc.)
- Faculty advisors for clubs (each club has one advisor, faculty can advise multiple clubs)
- Room reservations for club events
- Club budget and expense tracking

Your Tasks: 1. Create a complete ER diagram for this system 2. Convert your ER diagram to a normalized relational schema 3. Identify at least one design decision where you had multiple valid options and explain your choice 4. Write 3 example queries that your database should support (in English, not SQL)

Example Query Format: - “Find all students who are officers in the Computer Science Club” - “List all events scheduled for next week with their room reservations”

4) “Find the current budget of all clubs with their corresponded names”

“List all students names of the club N”

“List all clubs that are advised by the advisor A”

1 and 2) As seen in the corresponded file

3) I decided to design budget and expenses tracking as derived values because they are values calculated by a sum of a set of values.

In “budget” case, I suppose there is some starting value, so the ClubBudget attribute contains this value. Then, the budget tracking divided into ClubEarns and ClubExpenses multivalued attributes, so the total expenses could be calculated.

There, budget is calculated as starting budget + (ClubEarns + ClubExpenses). So the actual budget stored in CurrBudget

Total club expenses calculated as sum of all values in ClubExpenses and stored in CurrExpenses.